

This looks like an incredible final project! It combines everything you've learned into a complex, emergent ecosystem.

Here is a fully detailed, well-structured, and highly readable rewrite of the documentation to help you tackle the challenge.

Truth Arena: Build a Society That Beats the Lies

Welcome to the final challenge. You are putting together concepts from Conway's Game of Life, Neural Cellular Automata, and Boids into a single, cohesive simulation.

The Challenge

You are tasked with growing a society of truth cells on a grid. The environment contains food for growth, but it also contains liars—misinformation that spreads by touching your cells and flipping them. Once converted, a liar cell will continue to spread.

You cannot control the world directly. Instead, you must design a single local rule (a tiny brain) that every truth cell executes simultaneously every step. From this single rule, your society must emerge, grow, stick together, feed itself, and survive.

After 70 steps, your society is evaluated. The larger and more organized it is, the higher your score will be.

The Trap: Growth vs. Immunity

Growing everywhere as quickly as possible is a trap.

- A sprawling society becomes thin and hungry when it moves far from food.
- Weak and starving cells are easily converted by liars.
- A single liar can trigger a cascading failure that destroys the entire society.
- Huddling in a single safe spot provides immunity but results in a tiny society that scores poorly.
- You must balance spreading out to grab food with staying dense to maintain immunity.
- Immunity comes from having a sufficient number of truth neighbors.

Designing the Brain

Your final submission will be the weights of a small neural network (approximately 600 numbers). Every truth cell runs this network, reading its local window to decide its next action.

Complex behaviors like building walls or triggering alarms must naturally emerge from this local decision-making process, as there is no central controller.

What a Cell Senses

Sensor	Meaning
truth	Friendly cells nearby, which dictates your density and immunity.

Sensor	Meaning
liar	The nearby threat.
food	Locations where your cells can grow.
energy	How well-fed the current cell is.
memory	3 hidden numbers the cell can set for itself, used for signaling.

What a Cell Outputs

- A grow signal determining if the cell should attempt to spread.
- Updates to its 3 memory numbers.

Workspace & Files

File	Purpose
solution.py	Your workspace. Edit this to define the reward, search parameters, and your strategy NOTE.
truth.py	Read-only sandbox showing exactly what a cell senses.
check.py	Validator to run on your my_rule.npz file before submitting.
train.py	A no-code quick trainer alternative to solution.py.
render.py	Draws the replay gifs.
solution_nn.py	Bonus file to train a neural net from scratch using REINFORCE.
grow_nca.py	Bonus file to watch a from-scratch NCA grow a shape via gradient descent.
requirements.txt	Contains the two required libraries for installation.

How to Play

You will do all your work inside a single file: solution.py. You only need to edit two sections marked with >>> YOUR CODE <<<:

1. The Reward: Define your theory of the game and what makes a good society (e.g., size, density) to guide the training.
2. The Search: Define the training loop to search for good weights using a genetic algorithm.

The genetic algorithm works by keeping a group of rules, scoring them in the simulation, keeping the best ones, mutating them, and repeating the process.

Command Cheat-Sheet

- Install dependencies: `pip install -r requirements.txt`.
- Train your rule: `python solution.py` (Outputs `my_rule.npz`).
- Watch a replay: `python train.py --out my_rule --gif` (Generates `my_rule.gif`).
- Check submission: `python check.py my_rule.npz` (Must say PASS).

(Note: Python 3.9+ is required.)

Machine Learning Frameworks

You do not need TensorFlow or PyTorch for this challenge.

The cell's rule is a tiny network already written in plain numpy inside `truth.py`. Because there are no labels to learn from, you cannot use gradient descent or backpropagation. Instead, you use an evolutionary search to find the correct weights, which works universally without heavy frameworks.

(Advanced: If you insist on using your own framework, you can build a $21 \rightarrow 24 \rightarrow 4$ network, flatten the weights into `[W0, b0, W1, b1]` order, and inject them into `TruthRule().set_flat(your_numbers).save("my_rule")`.)

Scoring & Mandatory Rules

How You Are Scored

- Score is calculated at step 70.
- Score = society size × cohesion (How many truth cells are alive multiplied by how clumped together they are).

Rule 1: The Design Note

- You must update the NOTE string at the top of `solution.py` with 1-2 real sentences explaining *why* your society survives.
- Submissions lacking this note (or leaving it as "TODO") will be rejected by `check.py` and the host.

Rule 2: The Worst-Case Scenario

- The hidden test evaluates your rule across three liar-aggression tiers: easy, medium, and hard.
- Your final rank is determined by your performance on your weakest tier.

- You must build a robust rule that handles all difficulties.

Submission

- Submit only the my_rule.npz file containing your weights and note.

Emergent Strategies to Try

To win, you must invent clever strategies. Some ideas include:

- Membrane: Grow a dense, immune outer wall of truth cells to shield the inner core.
- Signalling: Utilize the hidden memory channels as an alarm system to alert neighbors of nearby liars.
- Smart Foraging: Spread toward food sources without letting the outer edge drop below the density required for immunity.
- Self-Healing: Detect gaps torn by liars and rapidly regrow them before the infection spreads inward.